The Scout/Fighter Fleet

Delegation and amalgamation under fog of war

October 30, 2026

Abstract

This note describes the second half of the `starship` maze: not one agent, but *three*, orchestrated by container delegation and amalgamation. A *scout* is the only agent that can perceive; it ranges ahead, maps the maze, retreats on contact with a monster, and reports. A *captain* is blind to the true maze — its entire state is a fog-of-war map it assembles from scout reports — yet it is the piece that must reach the exit, and it does so marching only on ground a scout has revealed. A *fighter*, blind as well, is sent by the captain to clear whatever blocks the way. The construction makes three claims precise. The permission model is three disjoint command registries. The amalgamation is a genuine semilattice join, not a tuple gather. And existential hiding — the captain reasoning over a store it cannot see past — is literal: the captain reaches the exit with roughly two-fifths of the maze still unknown to it. Everything is deterministic; two runs produce the same bytes, including a self-contained replay website the demo generates from the move trace.

1 The one idea

The single-agent maze (see `maze.tex`) proves one agent in a real perceive–reason–act loop. It does not exercise the showpiece's tagline — *orchestration by delegation and amalgamation* — which until now lived only in the scripted star-system bridge. The fleet closes that gap on the sharpest possible example: **partial observability**. The captain acts on a map it cannot see directly, only on what scouts report. That is existential hiding made visible rather than asserted.

A container is a pair (P, R): shapes P one may ask, and for each p a type $R\,p$ of permitted answers. A morphism (an open agent) is a pair $u : P \to Q$ (control flows *down*) and $f : (p : P) \to T(u\,p) \to R\,p$ (data flows *up*, contravariantly). A *worker* is a stateful morphism $\Delta S \otimes X \to Y$, where ΔS is the state object; it unfolds into a forward map that reads state and answers, and a backward map that splits into a writeback of new state and the answer sent up. The fleet is three workers composed: the scout and the fighter are workers the captain spawns, and the captain is itself a worker whose state is a fog map. Nesting is *existential* hiding: the captain sees a sub-agent's report, never its store.

2 Three agents, three registries

Each agent is a different sub-container of the *same* command-line surface (the `dungeon` CLI over the maze world). What distinguishes them is an allowlist — the *registry* — checked by the harness before any command runs. The registry *is* the permission grant, enforced by absence; there is no `bash`, no generic "run a command."

Agent	Registry	Perceives	Role
Scout	`{look, move}`	the true maze (its own `look`)	map it, retreat on contact, report
Fighter	`{move, attack}`	nothing — *blind*	clear the monster the captain names
Captain	`{move}`	only its `FogMap`	decide, and march to the exit

This is the star-system's "Science is read-only; Helm cannot fire; Weapons cannot steer" re-themed: the fighter cannot `look`; the captain cannot `attack`. A fighter that tried to `look` would be refused by the same gate that refuses a mutation with no reason, and the refusal would be recorded in the intent-log — the permission is visible in the audit, not merely claimed in a comment.

3 The fog is a semilattice, and the merge is its join

The captain's state is a `FogMap`: for each cell, what the fleet knows of its *terrain*. The subtle design point is that terrain and *occupancy* are different kinds of fact and must not share a type.

Terrain in a static maze is permanent, so knowledge of it only accumulates. The four values form a flat lattice with bottom $\bot$ = `unknown` and top $\top$ = `conflict`:

$$\texttt{unknown} \sqsubseteq \{\texttt{floor}, \texttt{wall}, \texttt{exit}\} \sqsubseteq \texttt{conflict}.$$

The three terrains are pairwise incomparable; the top exists only so the join is *total* — two scouts reporting different terrain for one cell join to `conflict`, which on an honest static maze never happens (a tested invariant). The cell join `joinKnown` is commutative, associative, idempotent, with `unknown` the identity and `conflict` absorbing; these are checked exhaustively over the five values.

`mergeFog` lifts the join cellwise. It is the **amalgamation** the fleet is built to demonstrate, and it is tested to be idempotent, commutative, associative, and *monotone* (merging only ever climbs the lattice — knowledge never decreases). This is the "real algebra" pole of the gold family: the treasury sums sub-balances up a chart of accounts; `org-chart` joins partial knowledge up a semilattice; the fleet does the latter. It is emphatically *not* the star-system captain's `tensorC`, which merely gathers two reports into a tuple.

Why occupancy is not in the lattice

A monster *dies* when the fighter clears it, so a cell's occupancy can retract — it is not monotone. Folding "monster" into the terrain enum (as a first draft did) would make a kill drop a cell `monster` → `floor` and the monotonicity claim would be a lie. So monsters live in a separate, revisable dictionary, and `mergeSighting` *retracts* a monster at a cell it now observes to be empty. Terrain joins monotonically; occupancy tracks the latest look.

4 The round is a dependent pair

The captain's control loop composes the scout and the fighter with sequential composition $\lhd$ — the dependent pair — using the vendored container combinators in `lib/algebra.ts`. One round:

```
seqC(scout, fighter).step(
  { tag: "explore" },                    // control DOWN: dispatch the scout
  (scoutReport) => {                      // the scout's position chooses the
    next prompt
```

```
    for (const s of scoutReport.sightings)  // data UP: amalgamate into the fog ...
      captainFog = mergeSighting(captainFog, s);   // ... this is the M1 join
    return scoutReport.blockedBy             // control DOWN again
      ? { tag: "engage", target: scoutReport.blockedBy.monsterId,
          path: pathOverFog(captainFog, fighterPos, adjacentTo(scoutReport.
    blockedBy.at)) }
      : { tag: "standby" };
  });
```

Read it as the type. $\triangleleft$ is the dependent pair $(p : P) \otimes (T(u\,p) \to R\,p)$: the scout's report is the value on which the fighter's prompt depends. The fighter's order cannot be *built* until the scout's report exists — that dependency is exactly why the composition must be $\triangleleft$ and not $\otimes$. The star-system captain's `tensorC(science, engineering)` owes both replies and lets neither depend on the other; here the whole point is that the second prompt is a function of the first answer. The amalgamation — the semilattice join of §3 — is the body of that function. Control flows down, data flows up, state (the fog) is threaded through the captain across rounds.

A soundness trap, avoided

The fighter's shape is a sum: `engage` or `standby`. Declaring the fighter's container as a *union of two fibres* would let TypeScript's conditional types distribute $\triangleleft$ over $+$, silently computing (scout $\triangleleft$ engage) + (scout $\triangleleft$ standby) — "decide then ask" — when what the round needs is scout $\triangleleft$ (engage + standby), "ask then decide." The fix is to give the fighter a *single* fibre with a union *shape*, `Fib<FighterShape, FighterReport>`, so the pair stays a single dependent step.

5 Existential hiding, literally

The captain is never handed the world. Its only state is the fog; its only input is scout reports. It explores — repeating the round — until a route to the exit exists *on what it already knows*, then it marches. Because it halts the instant the exit becomes reachable, large regions it never needed stay dark. On the shipped 21 × 15 maze (a perfect maze: one path between any two cells, hence ten dead ends that force the scout to backtrack), the captain reaches the exit with **136 of 315 cells still `unknown`** on its map. The closing screen of the demo prints the captain's final fog beside the true maze; the contrast is the point.

The scout, correspondingly, is a worker whose store — its own fog — is hidden. Within one `explore` dispatch it looks, steps toward the nearest frontier, and repeats up to five times, stopping early if its next step would be a monster (it reports `blockedBy` and holds position: "retreat" is handing control back, not walking home). The captain sees only the sightings it chose to report, never the scout's working map. The fighter, blind, cannot even see where it is: it *dead-reckons* its position from the orders it has executed, and reads its own hit points back from each attack's committed effect — the one thing a blind unit can still learn.

6 Two logs, three actors

Nothing about the audit changed; it simply now spans three agents. Every move and attack goes through the same gate as the single-agent maze: the registry is checked, a non-empty reason is required before the command runs, a write-ahead record is appended to the intent-log (Layer 2, the *why*) with a correlation id, and on commit the `dungeon` CLI appends to the ship's log (Layer 1, the *what*, no reason). The acting unit is recorded in both, so `agent:scout`, `agent:fighter`, and `agent:captain` appear as distinct actors. Reconciliation cross-checks the

two logs: an alarm is a committed mutation with no logged reason (it must never exist); a blocked action — a refusal — is visible only in Layer 2. On the shipped run, reconciliation reports all mutations matched and zero alarms: the captain moved the world only through reasoned, logged orders, and only ever onto ground a scout had revealed.

7 Determinism, and the replay

Every seam is seeded in the scripted path: the scout's and captain's searches explore neighbours in a fixed order, ids and the clock are injected stand-ins, and there is no randomness or wall-clock anywhere in the agents. `verify.sh` runs the fleet demo twice and diffs stdout, both logs, the move trace, *and* the generated website. The demo writes three artifacts: `moves.txt` (the moves as plain text), `trace.json` (every frame: the true positions and the captain's fog), and a self-contained `index.html` that animates the solve — the true maze with the captain's fog lifting over it as the scout reports come in, the fighter clearing monsters, and the captain marching out at the end. The website is a pure function of the deterministic trace, so it too is byte-identical across runs.

8 The leaf seam: scripted and model

The scout's decision at each step sits behind one interface, a *brain*, exactly as the single-agent maze puts the whole model behind a `ModelClient`. The default `scriptedScoutBrain` is a breadth-first step toward the nearest frontier — keyless and deterministic, which is what keeps the demo byte-identical. The `--live` brain hands a free model (Groq's OpenAI-compatible endpoint, reused unchanged from the single-agent maze) the one decision where judgement helps: at a *junction*, which of several productive directions to explore. In a corridor there is a single productive step and the model is not consulted; if the only productive step is onto a monster, the loop reports `blockedBy` as before; and a malformed or off-menu reply falls back to the BFS pick, so a weak model can slow the solve but never break it. The fighter is mechanical (it follows the captain's path and strikes) and the captain is the deterministic orchestrator, so both stay scripted — the scout is the only agent whose choices a model meaningfully makes. A live run uses the real id and clock seams, never the demo's seeded stand-ins: a live audit must carry true timestamps.

Run it: `deno task fleet`, then open `demo-out/fleet/index.html`. For the model-driven scout, `GROQ_API_KEY=... deno task fleet -- --live`.